



ICS3U Resource Book

Revised August 2013

3.1 Decisions and Relational Expressions

In order to make decisions, Java uses expressions of type `boolean`. Recall that there are two values of type `boolean`: `true` and `false`. Just as we can for other primitive types, we can create and use variables of type `boolean`.

Example 1

This program demonstrates creating and printing of boolean variables.

```
class BooleanOutput
{
    public static void main (String[] args)
    {
        boolean x = false;
        boolean y = true;
        System.out.println("x: " + x + "    y: " + y);
    }
}
```

This program will print

x: false y: true

One of the most common ways that we create `boolean` values is by evaluating a comparison of other types of values.

Example 2

- (a) The expression `2 < 3` has the value `true`.
- (b) The expression `-6.3 > 1.75` has the value `false`.

Expressions like the ones shown in this example are called *relational expressions* because their values depend on the relationship between other values. The example used the *relational operators* `<` and `>`. Java has a total of six relational operators. They are shown in the following table.

Relational Operator	Meaning
<code><</code>	is less than
<code><=</code>	is less than or equal to
<code>></code>	is greater than
<code>>=</code>	is greater than or equal to
<code>==</code>	is equal to
<code>!=</code>	is not equal to

Operators that are constructed from two characters must *not* have embedded blanks and must be in the order shown. The `==` operator is a frequent cause of errors in Java programs; people often omit the second equals sign. The operator is written as it is to distinguish it from the assignment operator.

Values of any of the primitive types can be compared in relational expressions. Values of type `boolean` can only be compared for equality or inequality; comparison of the relative sizes of `boolean` values is not meaningful.

Values of type `char` are ordered according to a *collating sequence*. Java's collating sequence is determined by the ordering of characters in the Unicode encoding system. A character that precedes another in the collating sequence is considered to be less than the other. Usually, comparison of characters is used with alphabetic characters and the results are what you would expect. The Unicode encodings for letters follow alphabetic order so that `'A' < 'B' < ... < 'Z'` and `'a' < 'b' < ... < 'z'`. One must, however, be careful with these comparisons as all upper case letters have encodings that are smaller than any of the lower case letters. In addition, accented letters such as `é`, `ü`, or `ç` have encodings that are larger than any unaccented letters. Life is never simple.¹

Luckily, it is rare that a programmer needs to be concerned with the details of Unicode encodings of characters. For most situations, all that you need to know is that

- `'0' < '1' < '2' < ... < '9'`
- `'A' < 'B' < 'C' < ... < 'Z'`

¹More details can be found in Appendix C.

- `'a' < 'b' < 'c' < ... < 'z'`
- A blank precedes any digit or letter.
- All digits precede all letters.
- All upper case letters precede all lower case letters.

Example 3

- (a) The expression
`'T' < 't'`
 has the value **true** because upper case letters have smaller Unicode encodings than lower case letters.
- (b) To evaluate the expression
`'#' < '+'`
 we need to know the Unicode encoding values of these characters. These values can be found in the table on page 600. From the table, we can see that the numerical value of the encoding of `'#'` is 35 while that of `'+'` is 43. Thus the expression has the value **true** because $35 < 43$.

Types can be mixed in relational expressions just as they can in arithmetic expressions. Any **char** values are compared to numerical values using their Unicode encoding values. Values of type **boolean** cannot be compared to values of any other type.

If arithmetic and relational operators are both included in an expression, the arithmetic operators all have a higher precedence than the relational operators.

Example 4

The expression $2.0 + 3 < 2 * 3$ would be evaluated as follows:

$$\begin{aligned}
 2.0 + 3 < 2 * 3 &\Rightarrow 2.0 + 3 < 6 \\
 &\Rightarrow 5.0 < 6 \\
 &\Rightarrow \text{true}
 \end{aligned}$$

Exercises 3.1

1. For each legal expression, state its value. For each illegal expression, state the reason that it is illegal.

- | | |
|-----------------------------|--------------------------------|
| (a) $(2 + (-5)) \neq 3$ | (b) <code>'m' <= 'p'</code> |
| (c) <code>'Q' == 'q'</code> | (d) <code>'*' < '*'</code> |
| (e) $8.23 \leq 8.2300$ | (f) $(7 / 3) = 2$ |
| (g) <code>false == 0</code> | (h) $(25 \% 4) \geq 1$ |

2. State, with reasons, what this program will print.

```
class BooleanVariables
{
    public static void main (String[] args)
    {
        boolean perhaps, maybe;
        perhaps = 4 < 5;
        maybe = -17 % 4 == 1;
        System.out.println("perhaps: " + perhaps);
        System.out.println("maybe:   " + maybe);
    }
}
```

3. For each expression, state whether it is true or false.

- | | |
|-------------------------------|-------------------------------|
| (a) <code>'q' < 'm'</code> | (b) <code>'G' > 'K'</code> |
| (c) <code>'a' < 'Z'</code> | (d) <code>'5' < 'v'</code> |
| (e) <code>'q' > '7'</code> | (f) <code>'9' < ' '</code> |
| (g) <code>'X' < 'y'</code> | (h) <code>'i' < 'I'</code> |

4. Determine the value of each expression.

- | | |
|-------------------------|--|
| (a) $17 / 3 < 17 / 3.0$ | (b) <code>'F' > 'B' + 3</code> |
| (c) $-6 \% 3 < 0$ | (d) <code>(int) 1.1 * 0.9 <= 0</code> |

(e) `(2 + 3 < 6) == true` (f) `(2 * 3 < 5) != true`

5. Because of the ways that Java's floating point values are stored, it may happen that numbers which should, in theory, be equal are, in fact, only approximately equal. Thus, it is not a good idea to test for exact equality of two floating point values. A better idea is to see if the numbers differ by less than some desired value. For example, given two `double` values `x` and `y`, the expression

`Math.abs(x - y) < Math.abs(1e-3 * x)`

will be true if and only if `x` and `y` differ by no more than 1/1000th of the value of `x`.

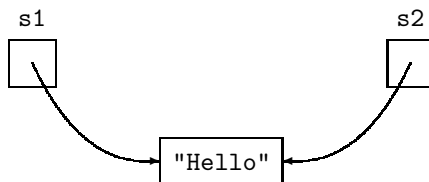
- (a) Write an expression that will be true if and only if `x` and `y` differ by no more than one one-millionth of the value of `x`.
- (b) Why is it a good idea to have the test dependent on the magnitude of one of the values being compared?

3.2 Comparing Strings

The relational operators used to compare primitive types should not be used to compare strings. The reason for this is based on the way that string objects are stored in Java. Suppose that we have written

```
String s1 = "Hello";  
String s2 = "Hello";
```

We can represent the result of executing these statements with a diagram.

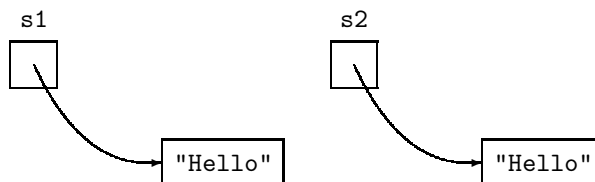


Because both `s1` and `s2` refer to strings with identical values, the Java compiler tries to save space by only allocating memory for one copy of the string `"Hello"` and having both `s1` and `s2` refer to this location.

On the other hand, if we were to write

```
String s1 = "Hello";  
String s2 = "He";  
s2 += "llo";
```

then both `s1` and `s2` will still refer to a string with the value `"Hello"` but now, because we have constructed the two strings in different ways, the Java compiler will probably not realize that they are the same² and will create *two* distinct copies of the string. The next diagram illustrates the resulting situation.



If we write the expression `s1 == s2`, Java evaluates this by comparing the values stored in `s1` and `s2`, *not* the strings to which `s1` and `s2` refer. If `s1` and `s2` refer to the same copy of a string (as in our first case), then `s1 == s2` will have the value `true` but if they refer to different copies of the same string (as in our second case), then `s1 == s2` will have the value `false` because `s1` and `s2` contain different memory references.

If we want to compare values of strings rather than the values of references to those strings, we should use methods. Java provides a number of methods for comparing strings. In this section, we will examine only two. In Chapter 9, we examine strings in more detail.

equals

To test whether or not two string variables are referring to equal string objects, we can use the method `equals` from the `String` class. The method tests two strings for equality and returns the value `true` if the strings are identical and `false` otherwise. This method is used in a manner that is

²We say “probably” because some Java compilers are better than others at detecting this sort of thing.

somewhat different from that used for the methods of the `Math` class that we saw in Chapter 2. If `s1` and `s2` are both references to strings, then the expression

```
s1.equals(s2)
```

will have the value `true` if and only if the values of the two strings are identical.

Example 1

Given the declaration

```
String s = "Same";
```

then

- (a) `s.equals("Same")` will return `true`
- (b) `s.equals("same")` will return `false` because `'S' ≠ 's'`
- (c) `s.equals("Same ")` will return `false` because of the blank in the string `"Same "`

compareTo

The *lexicographic order* of two strings is, essentially, their dictionary order; it is based on the collating sequence of the characters in the strings. To determine the lexicographic order of a pair of strings, we can use the method `compareTo` from the `String` class. In comparing strings, the method proceeds from left to right, character by character, as long as corresponding characters are identical. This process can terminate in one of three ways:

- The end of both strings is reached with all characters equal; the strings are identical.
- The end of one string is reached but the other is not; the shorter string precedes the longer string.
- A character in one string differs from the corresponding character in the other string; the string having the character earlier in Java's collating sequence precedes the other string.

The method `compareTo` returns an integer value that indicates the lexicographic ordering of two strings. If `s1` and `s2` are both references to strings, then the expression

`s1.compareTo(s2)`

has the value zero if `s1` is identical to `s2`, a negative value if `s1` precedes `s2`, and a positive value if `s1` follows `s2`.

Example 2

- (a) `"cab".compareTo("car") < 0` because `'b' < 'r'`
- (b) `"Car".compareTo("car") < 0` because `'C' < 'c'`
- (c) `"27".compareTo("186") > 0` because `'2' > '1'`
- (d) `"car".compareTo("cart") < 0` because `"car"` is only the first part of `"cart"`

The `compareTo` method can be used to implement any of the six relational operators (`<`, `<=`, `>`, `>=`, `==`, or `!=`) by writing expressions of the form `<string1>.compareTo(<string2>) <op> 0`

where `<op>` is one of the six relational operators. For example, if `s1` and `s2` are both references to strings, then the expression

`s1.compareTo(s2) <= 0`

has the value `true` if and only if the string `s1` precedes or is identical to the string `s2`.

Exercises 3.2

1. For each of the following pairs of strings, state, with reasons, which string precedes the other.
 - (a) `"cat"` and `"dog"`
 - (b) `"cat"` and `"Cathy"`
 - (c) `"X "` and `" X"`
 - (d) `"cab"` and `"CAR"`

- (e) `"XX"` and `"X X"`
- (f) `"XY"` and `"XY "`
- (g) `"375"` and `"84"`
- (h) `""` and `" "`

2. State the value of each expression.

- (a) `"one".equals("one ")`
- (b) `"two".equals("2")`
- (c) `"Three".equals("Three")`
- (d) `"four".equals("for")`

3. State whether the value of the expression is negative, zero, or positive.

- (a) `"for".compareTo("fore")`
- (b) `"fore".compareTo("force")`
- (c) `"force".compareTo("Force")`
- (d) `"Force".compareTo("Farce")`
- (e) `"Farce".compareTo("Fare")`
- (f) `"Fare".compareTo("far")`
- (g) `"far".compareTo("far")`
- (h) `"far".compareTo("far ")`

4. Suppose that `s1` and `s2` are variables of type `String`. Write an expression of the form `s1.compareTo(s2) <op> 0` that will have the value `true` if and only if the stated condition is true.

- (a) `s1` is identical to `s2`
- (b) `s1` precedes `s2`
- (c) `s1` is identical to or follows `s2`
- (d) `s1` does not follow `s2`

3.3 The if Statement

In performing a task, we often want to take one of two possible actions, depending on the conditions that exist at the time. For example

```
if you have homework, then
    you should do it,
otherwise
    you can go to a movie.
```

To accommodate this type of situation, Java uses an `if` statement having a form similar to the conditional English statement shown above.

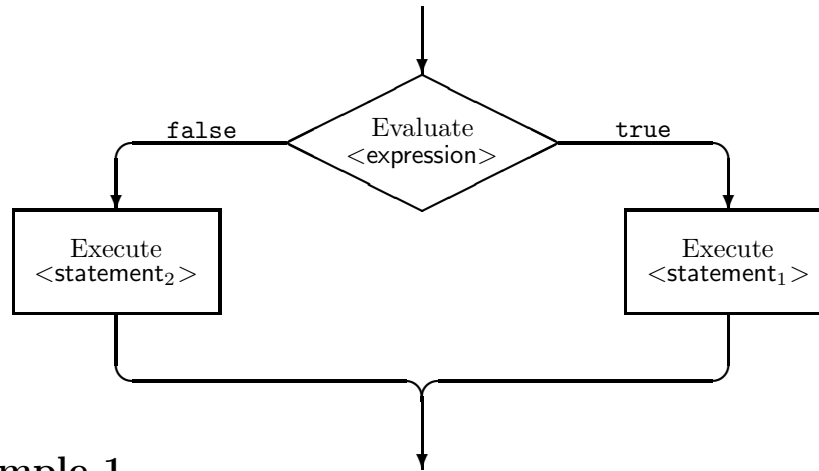
```
if (<boolean expression>)
    <statement1>
else
    <statement2>
```

The `if` statement's action is controlled by an expression that must be of type `boolean`. If `<boolean expression>` is true, `<statement1>` is executed; if `<boolean expression>` is false, `<statement2>` is executed. In either case, only one of the statements is performed.

Notice the parentheses that surround the `<boolean expression>`. These parentheses are *not* optional. Without them, the syntax of the statement is incorrect.

Notice also the way in which indentation is used in the `if` statement. Each of the alternatives is indented slightly past the first line. As we have said before, indentation is not required by the computer but, as the structure of programs becomes more complex, indentation becomes more and more important if we want to make the structure of our programs clear to a human reader.

Some people find it useful to illustrate the action of statements with a *flow chart*. As a program executes, we can imagine it travelling along the paths of the flow chart, following the direction of the arrows. Here is a flow chart for an `if` statement.



Example 1

This program compares two integers supplied by the user and prints an appropriate message.

```
class IfDemo
{
    public static void main (String[] args)
    {
        System.out.println("Please give one integer");
        int first = In.getInt();
        System.out.println("and a second");
        int second = In.getInt();
        if (first == second)
            System.out.println("The values are equal");
        else
            System.out.println("The values are not equal");
    }
}
```

If a user supplies two equal values to the program, the output will be

The values are equal

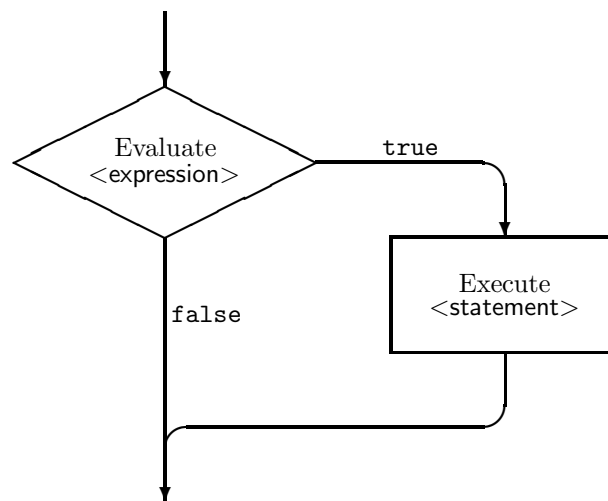
If the user supplies unequal values, the output will be

The values are not equal

Sometimes a special action is necessary only if a certain condition is true while no action is necessary if the condition is false. To accommodate this type of situation, Java allows us to write an `if` statement that has no `else` clause, as follows:

```
if (<boolean expression>
    <statement>
```

Here, `<statement>` will be executed if and only if `<boolean expression>` is true. If `<boolean expression>` is false, the program simply proceeds to execute the next statement in the program. The flow chart illustrates the action of this form of the `if` statement.



Example 2

The following statement warns a user if a value of `mark` indicates a failure on a test.

```
if (mark < PASSING_MARK)
    System.out.println("Mark indicates a failure");
```

If the value of `mark` is not less than the mark required to pass, no message will be printed.

If you always prefer to have `else` clauses in `if` statements, you can still obtain the effect of an `if` statement without an `else` clause by using an *empty statement*. An empty statement, as its name implies, has nothing in it (except for its terminating semi-colon). Despite its strange appearance (or lack of appearance), an empty statement can be used wherever any other type of statement can be used.

Example 3

Each of the following statements has the same effect: the value of `x` will be printed if and only if `x > 0`.

- (a)

```
if (x > 0)
    System.out.println("x = " + x);
```
- (b)

```
if (x > 0)
    System.out.println("x = " + x);
else
    ;
```
- (c)

```
if (x <= 0)
    ;
else
    System.out.println("x = " + x);
```

Frequently, when using `if` statements, we want to perform actions that require more than a single statement. To do this, we can create a *compound statement* or *block*, a group of zero or more statements enclosed by brace brackets.

Example 4

The following fragment checks on the sign of a value. If it is negative, the fragment prints a warning message and makes the value positive.

```
if (value < 0)
{
    System.out.println("Negative value set to positive");
    value = Math.abs(value);
}
```

Exercises 3.3

1. (a) Rewrite the following program using the indentation style shown in the text.

```
class VoterTest{public static void main(String[]args)
{final int VOTING_AGE=18;int age;boolean eligible;
age=In.getInt();if(age>=VOTING_AGE)eligible=true;
else eligible=false;if(eligible)System.out.println(
"Eligible to vote");else System.out.println(
"Not eligible to vote");}}
```

- (b) What would the program print if it were given input of 19?
 - (c) Replace the first `if` statement with a single assignment to the boolean variable `eligible`.
2. Write Java statements to perform each task.
 - (a) Add one to the value of `zeroCount` if the variable `total` has the value zero.
 - (b) Write an appropriate message depending on whether or not the value of the `int` variable `n` is a perfect square.
 - (c) Add one to the value of `pageCount` if the variable `lineCount` is greater than `pageLength`.
 - (d) Set the value of the boolean variable `leftSide` to `true` if the `int` variable `page` is even, and to `false` if `page` is odd.
 3. Write the simplest statement that has the same effect as the given statement.
 - (a)

```
if (x != y)
;
else
    System.out.println("Values are equal");
```
 - (b)

```
if (answer <= maxValue)
;
else
    System.out.println("Answer is wrong");
```

4. Write a program that prompts the user for two integers, determines whether or not the first number is a multiple of the second, and then prints both numbers and an appropriate message.
5. Write a program that reads a `double` value and prints both the number and its absolute value without using the built-in method `Math.abs`. The definition of the absolute value of x , written as $|x|$, is

$$|x| = \begin{cases} x & \text{if } x \geq 0 \\ -x & \text{if } x < 0 \end{cases}$$

6. Write a program that solves an equation of the form $ax+b=0$ by first prompting the user for the values of a and b , then solving the equation and printing the results. The program should take appropriate action if a is zero.

3.4 Boolean Operators

Just as arithmetic expressions can be acted upon by arithmetic operators to produce new arithmetic expressions, boolean expressions can be acted upon by boolean operators to produce new boolean expressions.

There are three commonly used boolean operators in Java: `!` (not), `&&` (and), and `||` (or).³ If `p` and `q` are both of type `boolean`, then

1. `!p` (not `p`) has the value `true` if and only if `p` has the value `false`.
2. `p && q` (`p` and `q`) has the value `true` if and only if both `p` and `q` have the value `true`.
3. `p || q` (`p` or `q`) has the value `true` if and only if at least one of `p` or `q` has the value `true`.

The results of applying the boolean operators can be summarized in *truth tables* as shown below.

p	q	!p	p && q	p q
true	true	false	true	true
true	false	false	false	true
false	true	true	false	true
false	false	true	false	false

³Appendix G discusses other, less commonly used, boolean operators.

Notice that the truth table for `p || q` has the value `true` not only if exactly one of them is true but also if both `p` and `q` are true. This may seem strange at first, but we often use the word “or” in this sense. For example, if one is asked, “do you take cream or sugar with your coffee?” it is reasonable to reply, “yes, I take both.” Taking “cream *or* sugar” includes the possibility of taking both.

If an expression contains more than one boolean operator, then, as with arithmetic expressions, precedence rules determine the order in which operations are performed. If we do not use parentheses, then the precedence order of the three boolean operations is

!
&&
||

As with arithmetic operations, the precedence rules can be overridden by using parentheses.

Example 1

```
!(true || false) && true  ⇒  !true && true
                        ⇒  false && true
                        ⇒  false
```

These operators can be mixed in expressions with other operators. The precedence order for the operators that we have seen up to now is

Operator	Operation
<code>++ --</code>	increment, decrement
<code>+ -</code>	unary plus, minus
<code>!</code>	boolean not
<code>(<type>)</code>	cast to <code><type></code>
<code>* / %</code>	multiplication, division, remainder
<code>+ -</code>	addition/concatenation, subtraction
<code>< <= > >=</code>	relational ordering
<code>== !=</code>	relational equality, inequality
<code>&&</code>	boolean and
<code> </code>	boolean or
<code>= += -= *= /= %=</code>	assignments

Operations with equal precedence are performed from left to right with the exception of assignment operations which (as we have already seen) are performed from right to left.

Example 2

```
4 < 5 || 'X' > 'Y'  ⇒  true || false
                     ⇒  true
```

In evaluating expressions involving `&&` or `||`, Java uses a technique known as *lazy evaluation* or *short circuit evaluation* or *conditional evaluation*. To show how this works, consider the expression `p && q`. Evaluating this from left to right, if `p` is false, then the entire expression must be false, no matter what value `q` has. Since the value of the expression is known without evaluating `q`, Java does not even look at `q`. Of course, if `p` is true, then Java must examine `q` to determine the value of the entire expression.

A similar situation arises with expressions of the form `p || q`. Here, if `p` is true, then the entire expression must be true so Java can again be lazy and not bother looking at `q`.

Example 3

Lazy evaluation can be quite useful in programming. Suppose that we want to perform some action if `a` is divisible by `b`. If we wrote

```
if (a % b == 0)
```

then an error would occur if `b` were zero and an `ArithmeticException` would be thrown. To guard against this, we could write

```
if (b != 0 && a % b == 0)
```

Now, if `b` were zero, the first half of the expression would be false and Java would not look at the second half of the expression so no error would occur.

Boolean expressions can often be simplified using a pair of rules known as *de Morgan's laws*, in honour of the logician Augustus de Morgan. To illustrate one of these laws, consider the statement “This book is boring and this course is useless.” For this statement to be true, both parts of it must be true. For the statement to be false, one or both of its parts must

be false. Thus, the negation of this statement can be written in the form “This book is not boring or this course is not useless.”

If we let **b** represent “This book is boring” and we let **u** represent “This course is useless”, then, from the above argument, the statement “it is not true that this book is boring and this course is useless” could be written as $!(b \ \&\& \ u)$. On the other hand, the statement “this book is not boring or this course is not useless” could be written as $!b \ || \ !u$. Since these statements are equivalent, then

$$!(b \ \&\& \ u) \text{ is equivalent to } !b \ || \ !u$$

This is one of de Morgan’s laws. The other is formed by interchanging the positions of $\&\&$ and $||$. As an example of the second form, the statement “I do not like green eggs or ham” means “I don’t like green eggs *and* I don’t like ham.” Using the symbol $\equiv$ to mean that two boolean expressions are equivalent, we can write the two laws as follows:

$$\begin{aligned} !(p \ \&\& \ q) &\equiv !p \ || \ !q \\ !(p \ || \ q) &\equiv !p \ \&\& \ !q \end{aligned}$$

Exercises 3.4

1. Evaluate each expression assuming that the following declarations have been made.

```
boolean p = true, q = false, r = false, s = true;
```

- | | |
|--------------------------------|-------------------------|
| (a) $!p$ | (b) $p \ \ q$ |
| (c) $p \ \&\& \ r$ | (d) $!(q \ \&\& \ s)$ |
| (e) $!q \ \&\& \ s$ | (f) $s \ \&\& \ !q$ |
| (g) $p \ \ !s$ | (h) $!p \ \&\& \ !q$ |
| (i) $s \ \ (!q \ \&\& \ r)$ | (j) $p == (q \ \ r)$ |

2. Simplify as much as possible.

- | | |
|-------------------|-------------------|
| (a) $!(a \leq b)$ | (b) $!(x \neq 0)$ |
|-------------------|-------------------|

- (c) `x < 5 && x < 8` (d) `a != 0 || a != 2`
 (e) `a > b || a == b` (f) `x > 0 && x <= -2`

3. Write a boolean expression that will be true if and only if the `int` variable `i` satisfies the condition $0 \leq i \leq 5$.

4. Use de Morgan's laws and the fact that `!!p` is equivalent to `p` to simplify each expression.

- (a) `!(p && !q)` (b) `!(!p && !q)`
 (c) `!(!p || !q)` (d) `!(x != 0 && !(y == 0))`

5. In addition to de Morgan's laws, there are a number of other laws that can be used to simplify boolean expressions. A useful list of equivalences is given in the following table. (The symbol $\equiv$ is used to mean that two boolean expressions are equivalent.)

<code>p && q</code>	$\equiv$ <code>q && p</code>	<code>p q</code>	$\equiv$ <code>q p</code>
<code>p && (q && r)</code>	$\equiv$ <code>(p && q) && r</code>	<code>p (q r)</code>	$\equiv$ <code>(p q) r</code>
<code>p && (q r)</code>	$\equiv$ <code>(p && q) (p && r)</code>	<code>p (q && r)</code>	$\equiv$ <code>(p q) && (p r)</code>
<code>p && p</code>	$\equiv$ <code>p</code>	<code>p p</code>	$\equiv$ <code>p</code>
<code>p && !p</code>	$\equiv$ <code>false</code>	<code>p !p</code>	$\equiv$ <code>true</code>
<code>p && true</code>	$\equiv$ <code>p</code>	<code>p false</code>	$\equiv$ <code>p</code>
<code>p && false</code>	$\equiv$ <code>false</code>	<code>p true</code>	$\equiv$ <code>true</code>
<code>p && (p q)</code>	$\equiv$ <code>p</code>	<code>p (p && q)</code>	$\equiv$ <code>p</code>
<code>!(p && q)</code>	$\equiv$ <code>!p !q</code>	<code>!(p q)</code>	$\equiv$ <code>!p && !q</code>
<code>!true</code>	$\equiv$ <code>false</code>	<code>!false</code>	$\equiv$ <code>true</code>
<code>!!p</code>		$\equiv$ <code>p</code>	

Use these relationships to simplify each of the following expressions as much as possible.

- (a) `(p && q) || (p && !q)` (b) `!p || (p && q)`
 (c) `!p && (!q || q)` (d) `p || !(p && q)`
 (e) `p && !p && !q` (f) `(p && !q) || p`
 (g) `!(!p && q) && (p || q)` (h) `!(p || !q) || (!p && !q)`

6. The *exclusive or* operator, which is sometimes written as $\oplus$, has the following truth table.

p	q	$p \oplus q$
true	true	false
true	false	true
false	true	true
false	false	false

Write a Java expression that is true if and only if $p \oplus q$ is true.

3.5 Nested if Statements

We have seen that we can use `if` statements to choose between two alternatives. Frequently, however, we want to choose from among more than two possibilities.

Example 1

If we want to write a statement based on the sign of a variable `x`, we could write three `if` statements.

```
if (x < 0)
    System.out.println("value is negative");
if (x > 0)
    System.out.println("value is positive");
if (x == 0)
    System.out.println("value is zero");
```

For any value of `x`, this solution involves three tests. We can increase the efficiency of the code by *nesting* the statements. This involves placing one `if` statement within another as shown in the next example.

Example 2

This improves the efficiency of the code in Example 1.

```
if (x < 0)
    System.out.println("value is negative");
else
    if (x > 0)
        System.out.println("value is positive");
    else
        System.out.println("value is zero");
```

Now, if *x* is negative, only one test is performed while if *x* is positive or zero, only two tests are performed. In no case is it necessary to perform three tests.

In the next example nesting is used in a slightly more complex way.

Example 3

The if statement shown below efficiently determines the largest among three values *x*, *y*, and *z* and assigns this value to *largest*.

```
if (x >= y)
    // y eliminated - largest must be either x or z
    if (x >= z)
        largest = x;
    else
        largest = z;
else
    // x eliminated - largest must be either y or z
    if (y >= z)
        largest = y;
    else
        largest = z;
```

There are many other ways that we could determine the largest of three values but the one shown here involves the least work.

In the examples that we have shown so far, we have been following the same indentation pattern that we introduced at the beginning of this chapter, aligning corresponding `if`'s and `else`'s and indenting two spaces past them for statements nested inside the `if`. If we have a long sequence of nested `if` statements, the resulting indentation can become excessive, leaving us little room on a line for our statements. To overcome this, we sometimes vary our indentation pattern.

Example 4

This statement prints a comment that depends on the value of the `char` variable `grade`.

```
if (grade == 'A')
    System.out.println("Excellent");
else if (grade == 'B')
    System.out.println("Good");
else if (grade == 'C')
    System.out.println("Average");
else if (grade == 'D')
    System.out.println("Fair");
else if (grade == 'E')
    System.out.println("Poor");
else
    System.out.println("Invalid grade");
```

As well as noting the indentation style, note that the last line covers the case of an invalid grade. You should always be sure that your code covers such possibilities.

A problem arises with nested `if` statements when one of the `if` statements has no `else` clause. The following segments illustrate the problem.

```
A:    if (p)
        if (q)
            System.out.println("Path A-1");
        else
            System.out.println("Path A-2");
```

```

B:   if (p)
        if (q)
            System.out.println("Path B-1");
        else
            System.out.println("Path B-2");

```

The indentation in segment A indicates that the **else** clause is associated with the second **if**. The indentation in segment B indicates that the **else** clause is associated with the first **if**. Since indentation is ignored by the computer, the two segments have the *same* structure. The problem of deciding which interpretation is correct is called the *dangling else* problem. In Java, interpretation A is the correct one. The general rule is to associate an **else** with the nearest available preceding **if**.

If we want to have the **else** associated with the first **if**, the preferred solution is to use brace brackets to “close off” the second **if** as shown in the next segment.

```

B*:   if (p)
    {
        if (q)
            System.out.println("Path 1");
    }
    else
        System.out.println("Path 2");

```

An empty statement provides an alternative solution to the problem.

```

B**:  if (p)
    if (q)
        System.out.println("Path 1");
    else
        ; // to "close off" the second if
    else
        System.out.println("Path 2");

```

Exercises 3.5

1. Simplify the following sequence by nesting so that the effect is the same but fewer comparisons are required.

```
if (temperature > maxTemp)
    System.out.println("Porridge too hot");
if (temperature < minTemp)
    System.out.println("Porridge too cold");
if (temperature >= minTemp && temperature <= maxTemp)
    System.out.println("Porridge just right!");
```

2. Consider the following statement.

```
if (age < minAge)
    if (income > minIncome)
        System.out.println("Accept");
    else
        System.out.println("Reject");
```

What will the statement print if

- (a) `age > minAge` and `income < minIncome`?
 - (b) `age < minAge` and `income < minIncome`?
3. Using nested `if` statements, write a single statement that prints the smallest value contained in the variables `a`, `b`, and `c`.
4. Write a statement that, if the value of the variable `item` is negative, adds its value to `negativeSum` but, if it is positive, adds its value to `positiveSum`. If, on the other hand, the value of `item` is zero, the statement should simply increase the variable `zeroCount` by one.
5. Assume that the following declarations have been made.

```
int year;
boolean isLeapYear;
```

Write a fragment that will assign `isLeapYear` the value `true` if `year` represents a leap year and `false` otherwise.

A year is *usually* a leap year if its number is a multiple of four. It is *not* a leap year, however, if its number is a multiple of 100 but not a multiple of 400. (As examples, 1900 was not a leap year but 2000 was a leap year.)

3.6 Choosing From Many Alternatives

Decisions with many possible choices can be handled using nested `if` statements but Java has another statement, the `switch` statement, that often makes choosing from many alternatives an easier task.

Example 1

To see how a `switch` statement works, suppose that we are writing a program that acts as a simple calculator. If at some point we have two numerical values (`operand1` and `operand2`) and a `char` variable `operator` with possible values `'+'`, `'-'`, `'*'`, or `'/'`, then the following fragment will print the value of the corresponding expression.

```
switch (operator)
{
    case '+': System.out.println(operand1 + operand2);
              break;
    case '-': System.out.println(operand1 - operand2);
              break;
    case '*': System.out.println(operand1 * operand2);
              break;
    case '/': System.out.println(operand1 / operand2);
              break;
}
```

In executing this statement, Java first determines the value of `operator`. Then, based on this value, it starts to execute the code inside the brace brackets, starting at the point indicated by the appropriate `case`. The `break;` statements cause Java to stop executing statements inside the `switch` and to jump down to the point just after the `switch`. For example, if `operand1 = 3`, `operand2 = 4`, and `operator = '*'`, Java would print the value of `3 * 4` and then exit the `switch`.

In the example, the final `break;` is not necessary but it is a good programming practice to put it there to make sure that the `switch` will

still work correctly even if, at some later time, an extra case is added to it. As others have noted, a little extra redundancy is often a good idea.

To guard against the possibility that no case matches the value of the expression in the `switch`, we can (and should) include a `default` case that contains statements to execute if there is no match. We can also have more than one case value associated with the same statements. Both of these ideas are illustrated in the next example.

Example 2

This `switch` statement assigns a grade based on a quiz that was scored out of five.

```
switch (score)
{
    case 5: grade = 'A';
           break;
    case 4: grade = 'B';
           break;
    case 3: grade = 'C';
           break;
    case 2:
    case 1:
    case 0: grade = 'F';
           break;
    default: System.out.println("Invalid score"
                               + " - grade of ? assigned");
            grade = '?';
            break;
}
```

In any `switch` statement, the expression that controls it must have some integer value (`byte`, `short`, `int`, `long`, or `char`). This expression is usually simply a variable but it can be an arbitrarily complex expression; the case values, on the other hand, must be constants. The order of the cases is arbitrary. If there is no `default` case and the value of the expression does not match any of the cases, then the statement does nothing (and Java gives no error message).

Exercises 3.6

1. Rewrite the following using the indentation style shown in the text.

```
switch(month){case 1:case 3:case 5:case 7:case 8:case 10
:case 12:length=31;break;case 4:case 6:case 9:case 11:
length=30;break;case 2:if(isLeapYear)length=29;else
length=28;break;}
```

2. (a) In Example 1, what would the fragment print if `operand1 = 8`, `operand2 = 2`, and `operator = '-'`?
(b) What would be printed with these values if there were no `break`; statements in the `switch` statement?
3. A sequence of six tests, all scored out of 100, are to be given different weightings in determining a final mark. Write a program fragment that computes the appropriate weighted score for one test. The fragment should first read values of `testNumber` and `score`. Using a `switch` statement, it should then compute and print the appropriate value of `weightedScore` using the weightings given in the following table.

Test Number	Weight
1	10%
2	20%
3	20%
4	15%
5	15%
6	20%

For example, input of

3

27

should produce output of

A score of 27 on test 3 gives a weighted score of 5.4

4. Write a program fragment that prints, as a word, the value of the last digit of the `int` variable `number`. For example, if the value of `number` is 547, the fragment should print

The last digit of 547 is seven.

5. Write a program that reads a date in numeric form as a year followed by a month followed by a day and then prints the date as one might on a cheque. For example, input of

2000

7

1

should produce output of

July 1, 2000

3.7 Avoiding Errors and Debugging

Avoiding Errors

1. Because of the way that floating point values are stored, it may happen that values that should be exactly equal are, in fact, only approximately so. Thus, it is not a good idea to test for exact equality of expressions that contain floating point values. A better idea is to see if the values differ by less than some quantity. To see if x and y are sufficiently close, we might use a test of the form $|x - y| < \epsilon$ where ϵ (the Greek letter epsilon) is some small positive quantity. A problem with this is that it applies the same test for closeness to both small and large numbers. While we might consider small numbers to be “close” if they differ by 0.001, we may consider very large numbers to be “close” if they differ by only 100. To handle such differences, it is a good idea to make our test of closeness depend on the magnitude of the quantities being examined. A good scheme that employs this principle is to use a test of the form $|x - y| < \epsilon \times \max(|x|, |y|)$.
2. It is legal to embed an assignment inside a condition. As an example, an `if` statement could begin with

```
if ((x = y + z) > 0) ...
```

Here the expression `(x = y + z) > 0` first assigns to `x` the value of

$y + z$ and then tests the result to see if it is positive. We sometimes refer to the assignment as a *side-effect* of the evaluation of the condition. Side-effects in programming, like side-effects in taking pharmaceutical drugs, are usually undesirable. Generally speaking, we recommend that expressions involving side-effects be avoided. For the given example, if you later choose to eliminate the `if` statement, you may inadvertently also eliminate the assignment. Usually it is best to write code that may be a little longer than is necessary in order to make your intentions clear. The example here could be replaced by

```
x = y + z;  
if (x > 0) ...
```

3. Be very careful about your use of semi-colons. One extra semi-colon in the wrong place can completely destroy the logic of a program. For example, the fragment

```
if (x < 0);  
    System.out.println("Value is negative");
```

will *always* print the message “Value is negative”, no matter what value `x` actually has. The problem here is the semi-colon at the end of the first line. Using our indentation scheme, Java interprets the fragment as

```
if (x < 0)  
;  
    System.out.println("Value is negative");
```

The extra semi-colon is considered to be an empty statement which is executed if `x` is negative. This terminates the `if` statement. No matter what value `x` has, the statement that prints the message is executed.

4. For statements to be treated as a block, they must be enclosed in brace brackets. Consider the following example.

```
if (value < 0)  
    System.out.println("Negative value made positive");  
    value = -value;
```

The indentation indicates that both statements should be executed if the condition is true but the compiler ignores indentation. Without brace brackets surrounding the two indented statements, the second one is not part of the `if` statement. The fragment will make negative values positive but it will also cause positive values to be made negative.

Debugging

1. The most common error committed by beginning Java programmers using `if` statements is to write `=` when `==` is required. The error will usually be detected as a syntax error by the compiler. The error message that it produces is (usually): **incompatible types**. To explain this, consider the statement that begins with

```
if (n = 5) ...
```

where `n` is of type `int`. Inside the parentheses is an assignment, an expression whose type is that of the variable on the left side of the assignment. In the example shown here, the type of the assignment expression is `int` because `n` is of type `int`. Java, of course, is expecting a boolean expression (like `n == 5`). Since the type that it finds is not what it wants, it complains.

2. Another frequent error with `if` statements is the failure to take some action for every possible eventuality. The Java compiler is sometimes clever enough to spot this sort of problem so that a fragment such as

```
int m;
int n = In.getInt();
if (n > 0)
    m = 1;
else if (n == 0)
    m = 2;
System.out.println(m);
```

will, during compilation, produce the error message “**variable `m` may not have been initialized**”. The problem here is that `m` will not be given a value if `n < 0`. Unfortunately, sometimes the compiler is a bit over-zealous in flagging possible errors of this type. For example, the fragment

```

int m;
int n = Math.abs(In.getInt());
if (n > 0)
    m = 1;
else if (n == 0)
    m = 2;
System.out.println(m);

```

will still produce the same message even though it is now impossible for `n` to be negative. The easiest way to get past this problem is to initialize `m` to zero where we declare it (in the first line of the fragment).

3. In mathematics, we often restrict the range of a variable x by writing expressions like $a < x \leq b$. Writing similar expressions in Java causes an error. To see what is wrong with such expressions, suppose that we were to try to evaluate `a < x <= b` as Java might do. Working from left to right, we would begin by evaluating `a < x` giving a boolean result (`true` or `false`). When we then attempted to complete the evaluation, we would be left trying to compare a boolean value with `b`, producing an error. Java's compiler is smart enough to spot such problems, producing an error message during compilation. The solution to the problem is to rewrite the expression in the form `a < x && x <= b`.
4. Another mathematical shortcut that produces an error in Java is an expression of the form $x, y > 0$. Trying to write this in Java either as it is or in the form `x && y > 0` is a mistake. To evaluate `x && y > 0`, Java would first have to evaluate the sub-expression `y > 0` (because `>` has higher precedence than `&&`) but then the expression would reduce to either `x && true` or `x && false`. For `x` having a numerical value, either of these expressions would be nonsense in Java. Again, the compiler will catch such an error and refuse to compile a program containing it. The expression should be replaced by `x > 0 && y > 0`.
5. If a program that contains `if` or `switch` statements is running but producing incorrect results, the error may be caused by the program executing incorrect statements because the logic is wrong. To determine which path a program is actually taking, insert tracing statements in every possible branch of the program. For example, to trace a program containing a segment of the form

```

if ( ... )
    if ( ... )
        <statement1>
    else
        <statement2>
else
    <statement3>

```

tracing statements should be inserted at each of the points at which <statement₁>, <statement₂>, and <statement₃> are shown.

6. Tracing statements used for debugging can be turned on and off easily using `if` statements. To do so, create a boolean variable `DEBUG` that is set to `true` when debugging and `false` otherwise. Then place all tracing statements inside `if` statements of the form


```
if (DEBUG) ...
```

Exercises 3.7

1. (a) In the section on avoiding errors, we suggested that a good test for the closeness of two floating point variables x and y was to see if $|x - y| < \epsilon \times \max(|x|, |y|)$. If `x` and `y` are two `double` variables and `EPSILON` is a `double` constant representing ϵ , write a Java expression that will be true if and only if `x` and `y` satisfy this test.
 - (b) For the values of `x`, `y`, and `EPSILON` that follow, state whether the expression in part (a) would be true or false.
 - i. `x = 123.48`, `y = 123.77`, and `EPSILON = 0.01`
 - ii. `x = 0.000456`, `y = 0.000457`, and `EPSILON = 0.001`
 - iii. `x = 8.7658`, `y = 8.7651`, and `EPSILON = 0.001`
 - iv. `x = 12626.7`, `y = 12539.2`, and `EPSILON = 0.01`
2. Assuming that `x`, `a`, and `b` are `double` variables and `a < b`, write the simplest Java expression that will be true if and only if
 - (a) `x` is between `a` and `b`
 - (b) `x` is outside the interval between `a` and `b`
 - (c) `x` is either less than `a` or less than `b`

(d) `x` is greater than both `a` and `b`

3. Examine the following fragment and then answer the questions that follow it.

```
if (x < 0)
    System.out.println("invalid");
else;
    System.out.println("valid");
```

- (a) For what value(s) of `x` will the word `valid` be printed? Justify your answer.
- (b) Rewrite the fragment so that the word `valid` is printed only if $x \geq 0$.

4. Suppose that a program contains the following statement.

```
if (a > 0)
    if (b < 0)
        System.out.println(1);
    else
        System.out.println(2);
else
    System.out.println(3);
```

Under what circumstances will the statement print

- (a) 1? (b) 2? (c) 3?

3.8 Review Exercises 3

1. Assume that the following declarations and assignments have been made:

```
int age = 16;
int height = 175;
int weight = 70;
char sex = 'M';
boolean healthy = true;
```

Using these values, evaluate each expression.

- (a) `age >= 16 && healthy`
- (b) `!(weight <= 75) && height >= 180`
- (c) `age > 10 || sex == 'F' && height < 170`
- (d) `!(height < 160) && (weight < 60)`

2. For each legal expression, state its value. For each illegal expression, state the reason that it is not legal.

- (a) `!(3 < 4.5)`
- (b) `'r' < 'q'`
- (c) `!(17 / 5 = 3.4)`
- (d) `'G' != 'g' || 1 + 2 <= 3`
- (e) `5 < Math.abs(3-2*5) < 10`
- (f) `'6' - '2' == '4'`

3. Given three strings `s`, `t`, and `u`, write a fragment that would print the string that would be first if they were to be printed in lexicographic order.

4. In Question 3, the ordering of the strings is not necessarily the order in which they would be printed in a dictionary. Explain.

5. Study this program and then answer the questions that follow it.

```
class ChipChoice
{
    public static void main (String[] args)
    {
        System.out.println("Preferred flavour?");
        char flavour = In.getChar();
        System.out.println("Preferred style?");
        char style = In.getChar();

        System.out.print("The client preferred ")
        if (flavour == 'B')
            System.out.print("bar-b-que");
        else if (flavour == 'V')
            System.out.print("vinegar");
        else
            System.out.print("other");
        if (style == 'C')
```

```

        System.out.print(", crinkled");
    else if (style == 'R')
        System.out.print(", regular");
    System.out.println(" chips.");
}
}

```

What would the program print given each set of data as input?

- | | |
|-------|-------|
| (a) V | (b) B |
| C | R |
| (c) V | (d) B |
| T | C |
| (e) C | (f) v |
| V | r |

- How would you modify the program of the previous question so that it would accept both upper case and lower case letters in the responses from the user?
- Rewrite as a single `if` statement in as concise a form as possible. Assume that `flag` is of type `boolean` and `n` is of type `int`.

```

if (true)
    if (!flag)
        n = 1;
    else
        n = 0;

```

- Rewrite using nesting.

```

if (p && q)
    System.out.println("Both true");
if (p && !q)
    System.out.println("Only first true");
if (!p && q)
    System.out.println("Only second true");
if (!p && !q)
    System.out.println("Neither true");

```

- Consider the following statement.

```

if ((a <= b && b <= c) || (a >= b && b >= c))
    System.out.println("Values in order");
else
    System.out.println("Values out of order");

```

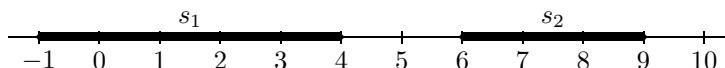
- (a) Describe in a few words the effect of the statement.
 - (b) Write a statement that has the same effect but uses neither `&&` nor `||`.
10. In the country of Rahmania, the cost of mailing a letter is 40 sinas for letters up to 30 g, 55 sinas for letters over 30 g and up to 50 g, 70 sinas for letters over 50 g and up to 100 g, and then an additional 25 sinas for each additional 50 g or part thereof. Write a program that prompts the user for a mass and then gives the cost of mailing a letter having that mass.
 11. One way of giving a direction is to simply use one of the letters N, E, S, or W indicating the nearest compass point. Another way is to give a bearing as a number from 0 to 359. A bearing of 0 corresponds to N, a bearing of 90 corresponds to E, and so on. Write a program that prompts the user for a bearing from 0 to 359 and prints the corresponding letter of the compass point nearest to that bearing. For example, input of 73 should produce output of E. Bearings half way between two compass points should be taken to be either N or S, as appropriate. Your program should make decisions efficiently.
 12. Suppose that two line segments on a number line are represented by the values of their endpoints: segment s_1 is represented by `left1` and `right1` while segment s_2 is represented by `left2` and `right2`. For example, the values


```

left1 = -1
right1 = 4
left2 = 6
right2 = 9

```

 would define the segments shown in the diagram.



Write expressions that will be true if and only if

- (a) s_1 is longer than s_2 .
- (b) s_1 is entirely contained within s_2 .
- (c) s_1 and s_2 have no points in common.
- (d) s_1 and s_2 overlap partially (but not completely).

Projects

13. Write a program to compute an employee's weekly pay and produce a pay slip showing name, gross pay, deductions, and net pay. The program should first prompt the user for:
 - (a) family name
 - (b) given name
 - (c) hourly rate of pay
 - (d) number of hours worked that week (Any hours over 40 are paid at double the normal hourly rate.)
 - (e) a letter indicating the employee's tax category
 - A: no tax deduction
 - B: tax is 10% of gross pay
 - C: tax is 20% of gross pay
 - D: tax is 29% of gross pay
 - E: tax is 35% of gross pay
 - (f) either a Y or an N to indicate whether or not the employee wants \$20 deducted from the weekly pay as a contribution to the United Way Charity
14. Write a program that makes change for amounts less than one dollar. Input to the program should be a positive integer less than 100, representing an amount of money, in cents. Output should be the original amount of money together with a set of coins (quarters, dimes, nickels, cents) that could make up that amount. The program should produce change containing the minimum number of coins required for the given amount. The output should be in a natural, non-stilted form. For example, input of 58 should produce output something like

58 cents: 2 quarters, 1 nickel, and 3 cents.

rather than

58 cents: 2 quarters, 0 dimes, 1 nickels, 3 cents.

15. The government of Simpleton has devised what it thinks is an easy income tax system, but its citizens still need help. They have commissioned you to write a program to ask a citizen a few simple questions and compute the tax that is payable or the refund that is due. You must write your program as clearly as possible so that the government can verify it easily. Your program should first ask a citizen for his/her income (`income`), housing cost (`houseCost`), number of children (`totalChildren`), and number of children that are in school (`schoolChildren`). It should then compute and print the tax payable or the refund due. The tax rules are as follows. The Simpleton tax rate is 18% but citizens are not taxed on the first \$10 000 of income unless they pay more than \$8 000 for housing. For every child, a Simpleton citizen gets a \$500 tax reduction, or \$1 000 if the child is in school. This reduction never results in citizens getting refunds unless their housing costs are less than \$6 000 and they have more than two children, at least one of whom is in school. Finally, if the tax payable is more than \$2 000, then it is increased by an additional 15% surtax.